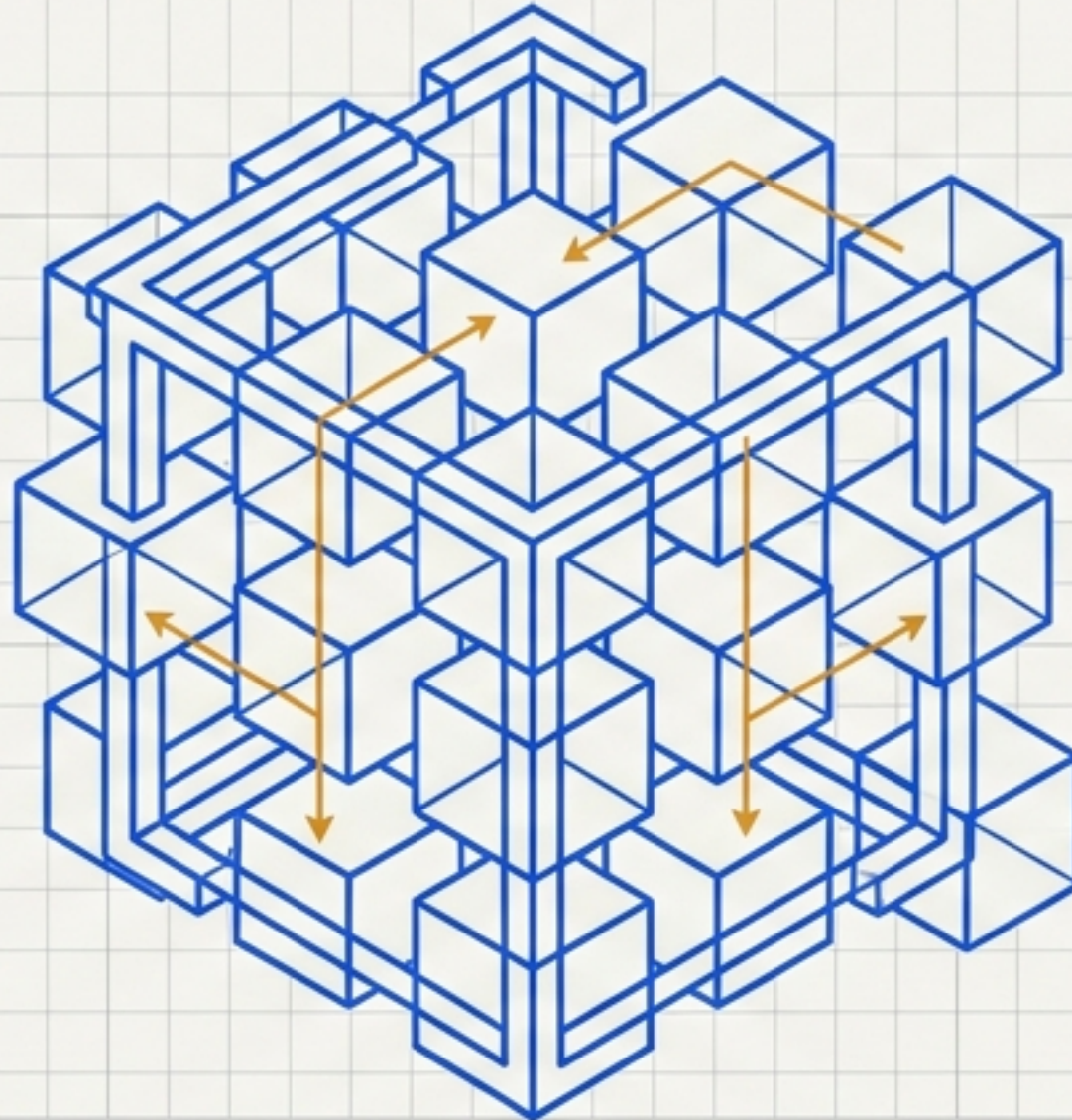


SOEN 201: Mastering Object-Oriented Analysis & Design

A visual study guide for concepts, principles, and UML relationships.



Focus: Core pillars and CAT-specific visual notations.

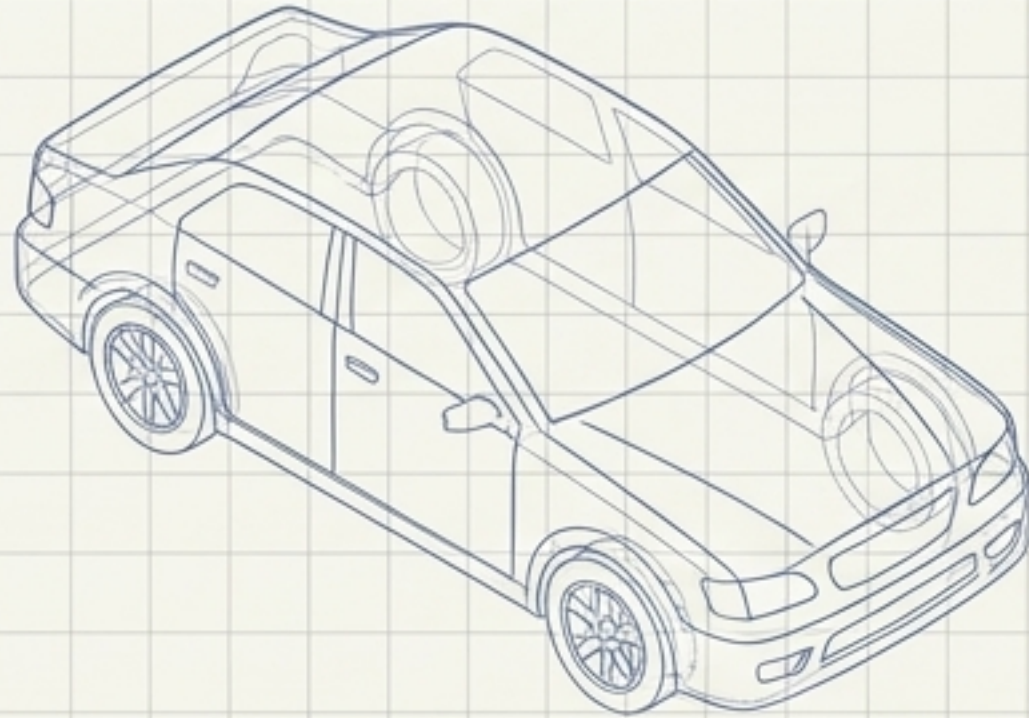
Programming paradigms evolved to tightly bind data and behavior

Procedural Programming	Modular Programming	Object-Oriented Programming
Focus: Procedures (functions) Data Binding: Separated Core Question: What steps should the program take?	Focus: Independent modules Data Binding: Loose Core Question: How do we break the program into smaller pieces?	Focus: Interrelated objects Data Binding: Tightly bound (Data Abstraction) Core Question: What objects exist and how do they interact?

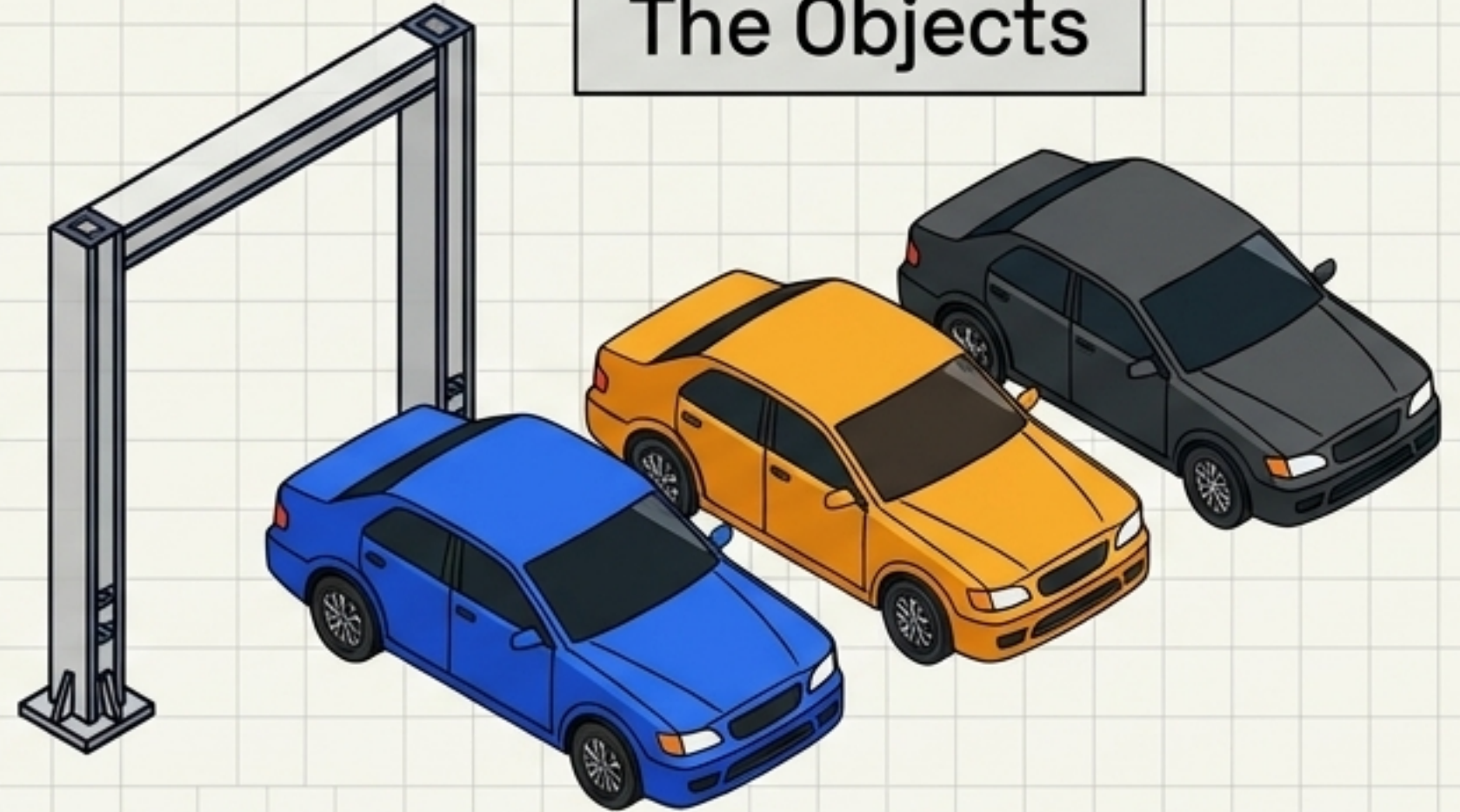
Classes act as theoretical blueprints while objects represent concrete instances

Factory Model

The Class



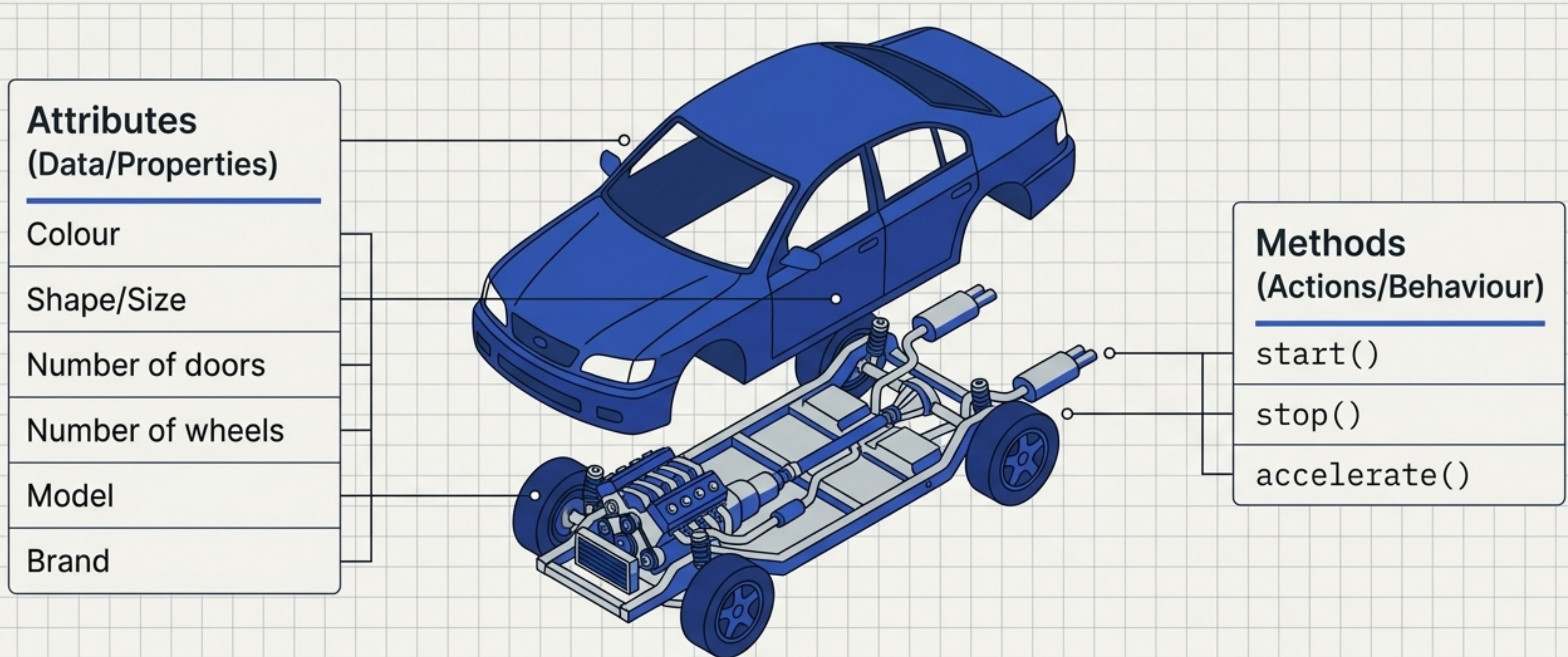
The Objects



- Blueprint or template
- Occupies NO memory for data

- Concrete instances
- Occupies memory

Every object encapsulates specific attributes and executable methods



```
Car myCar = new Car();
```

Instantiating a single object with unique attribute values.

Five core pillars govern object-oriented analysis and design

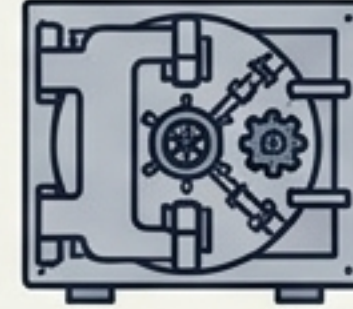
Abstraction

Hiding complexity



Encapsulation

Bundling and protecting data



Modularity

Independent, maintainable units



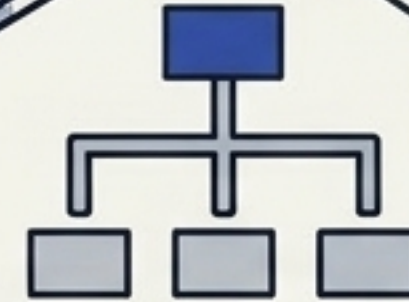
Polymorphism

Multiple functional responses



Inheritance

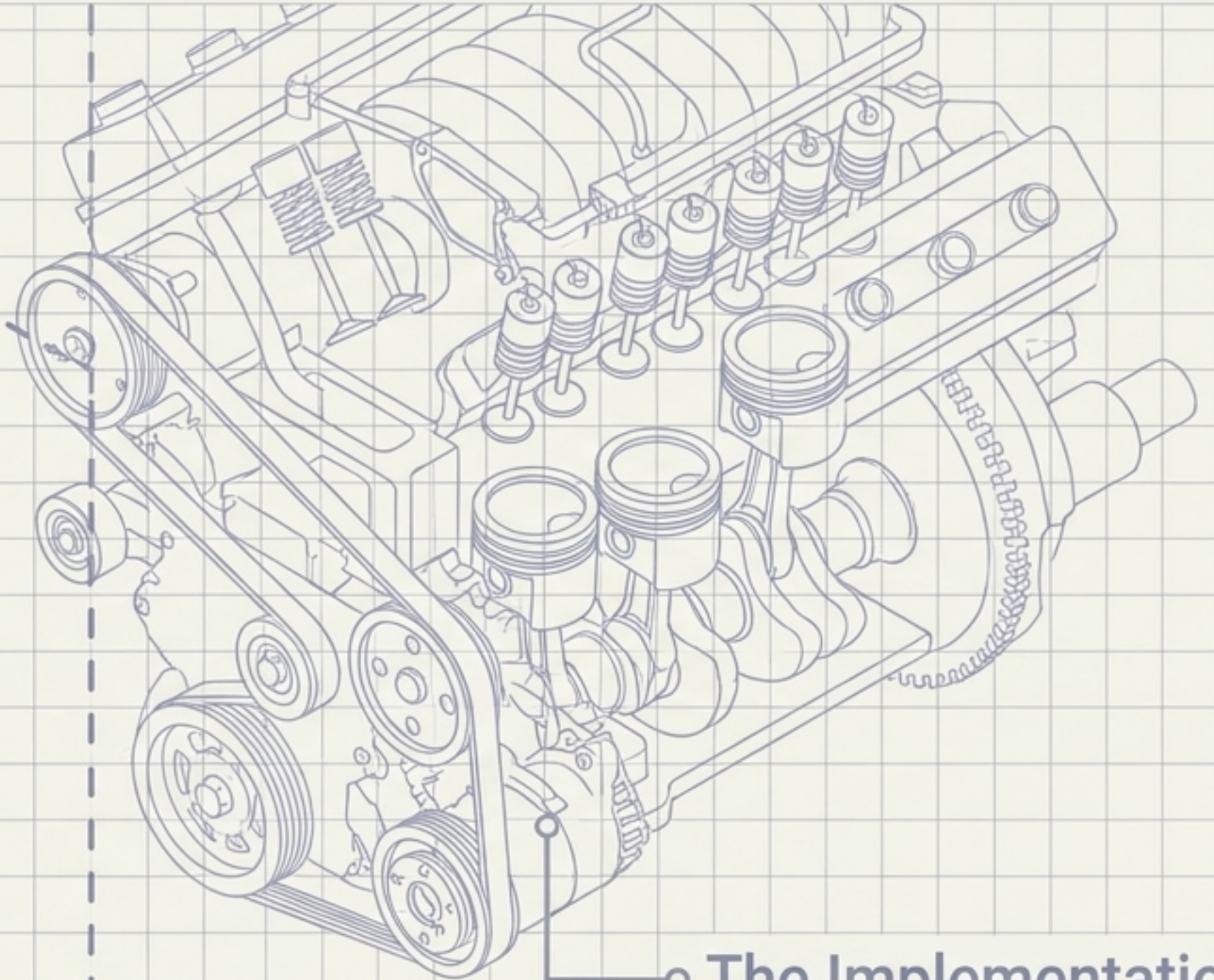
Parent-child attribute sharing



Abstraction hides internal complexity behind a simple interface

The Interface

Focus on what the object does.



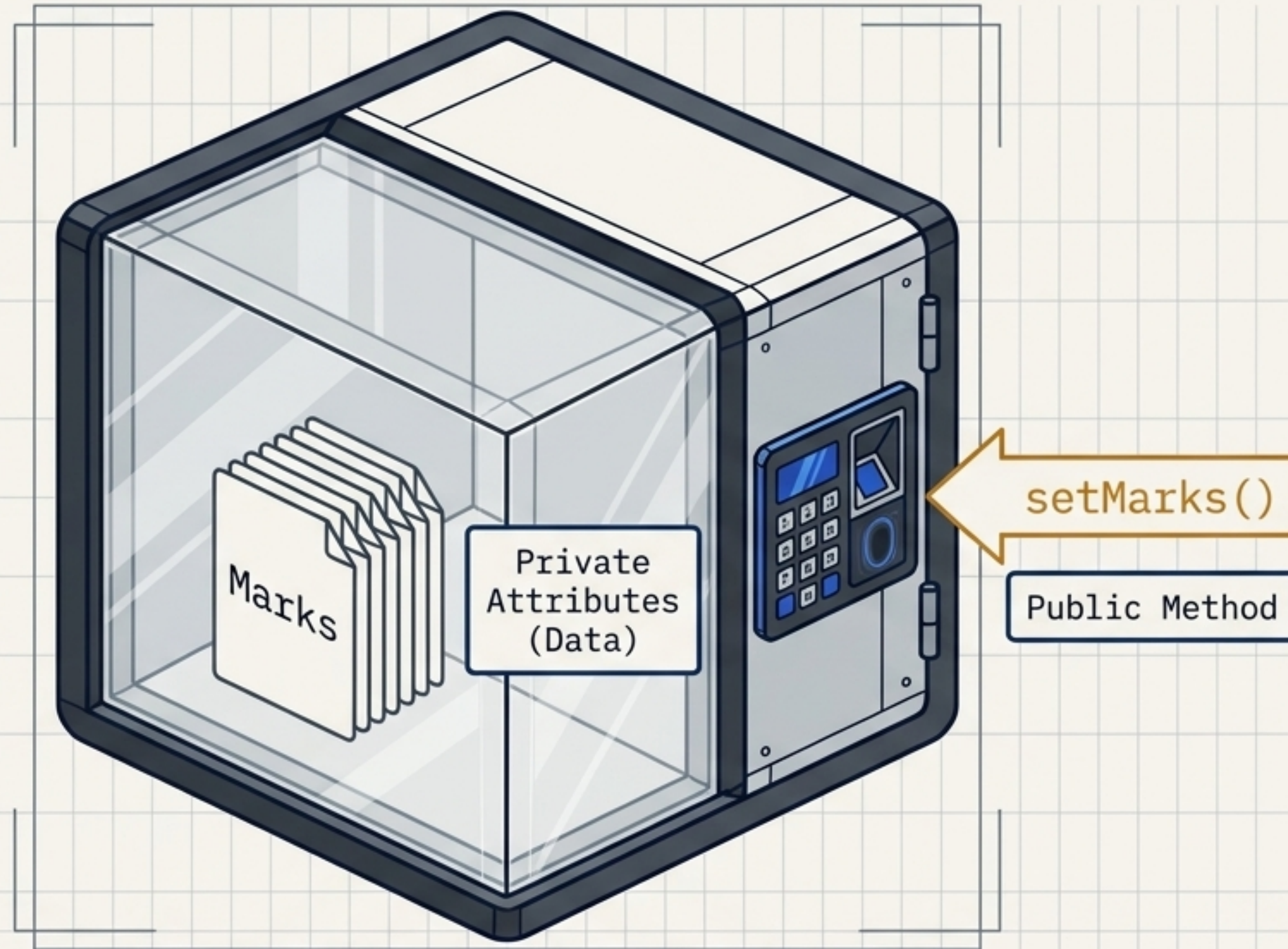
The Interface

Focus on what the object does.

The Implementation

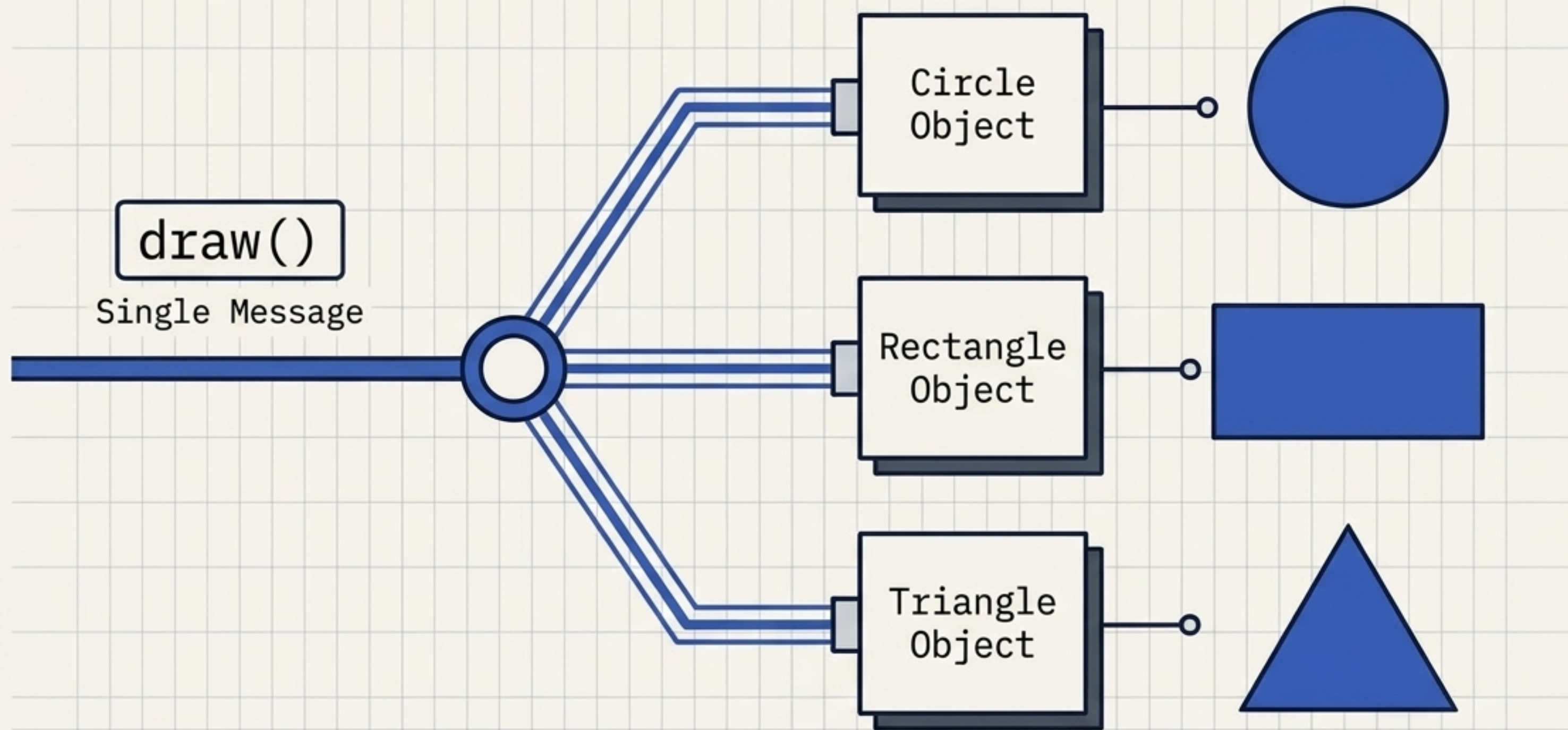
Do not need to know the internal engine details.

Encapsulation bundles data and controls access through public methods



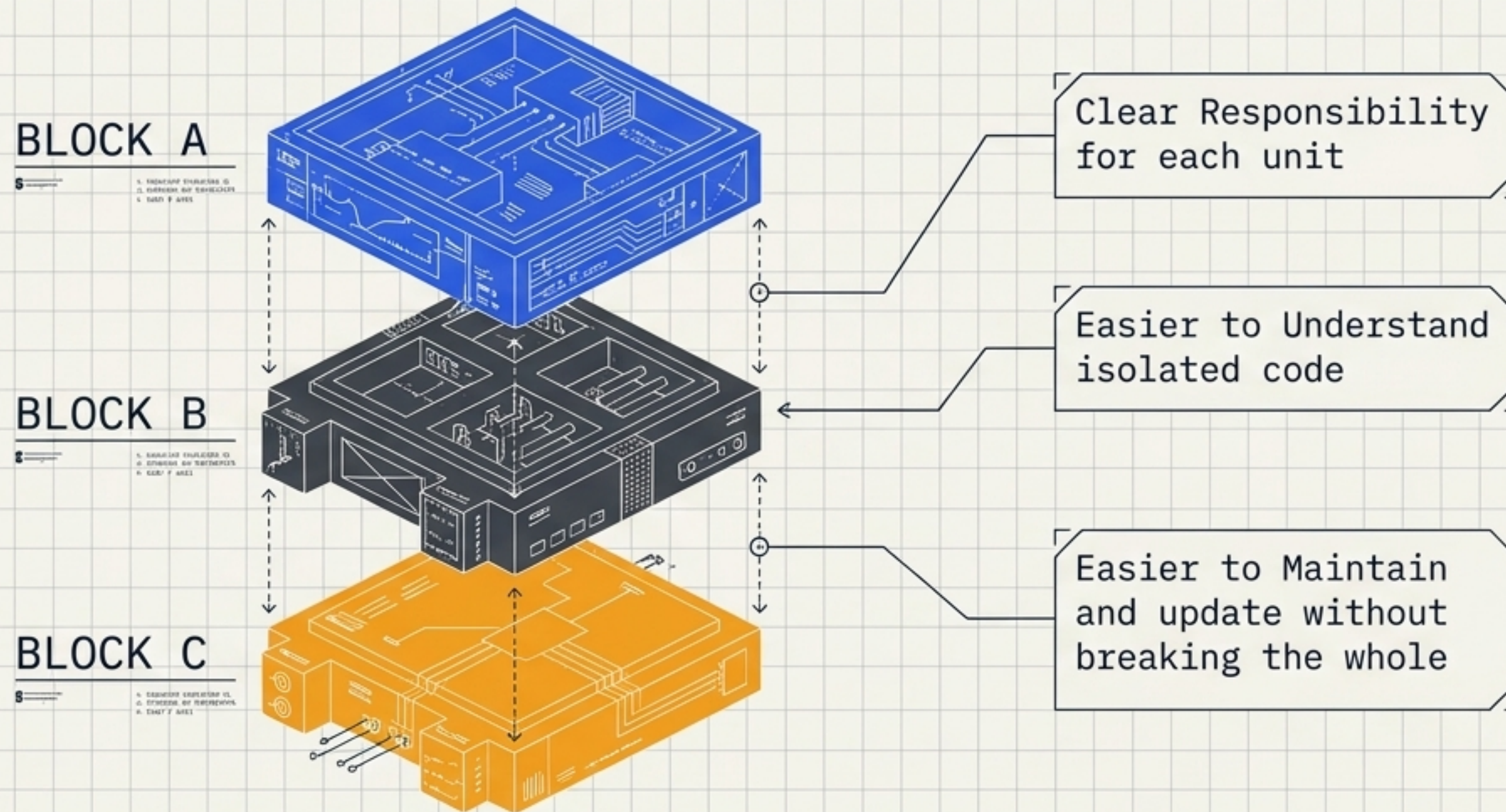
Other code cannot touch the marks directly; it must call the method, which validates the input and protects data integrity.

Polymorphism allows different objects to execute the same command uniquely



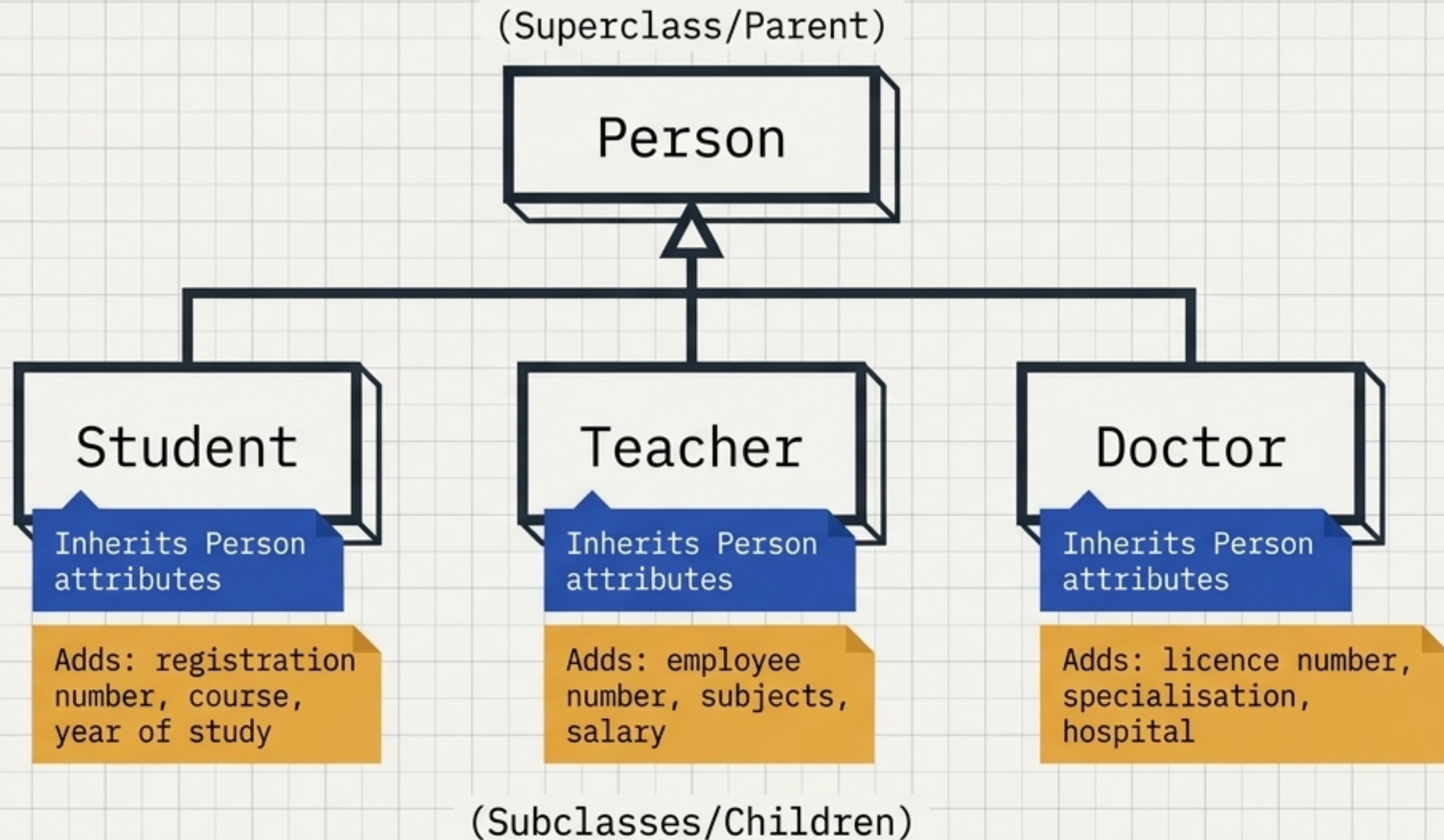
Different objects respond to the exact same message in their own distinct ways.

Modularity breaks complex systems into independent, manageable units

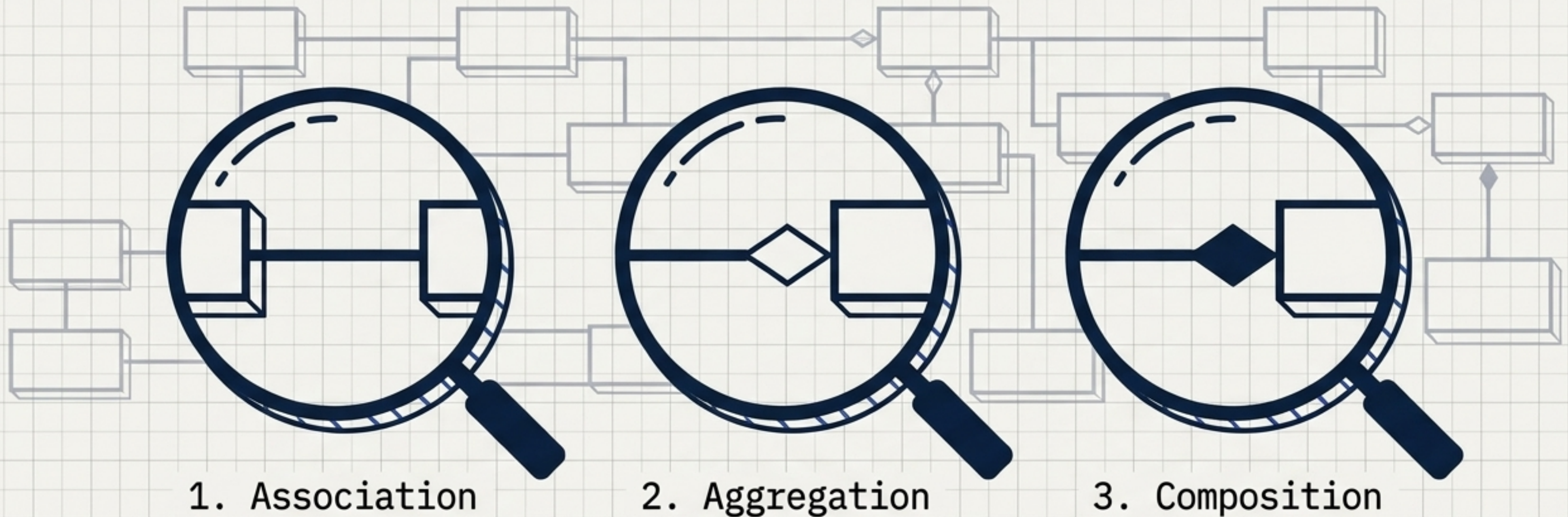


Breaking a program into smaller, independent classes prevents cascading failures.

Inheritance establishes an 'is-a' hierarchy to share common attributes

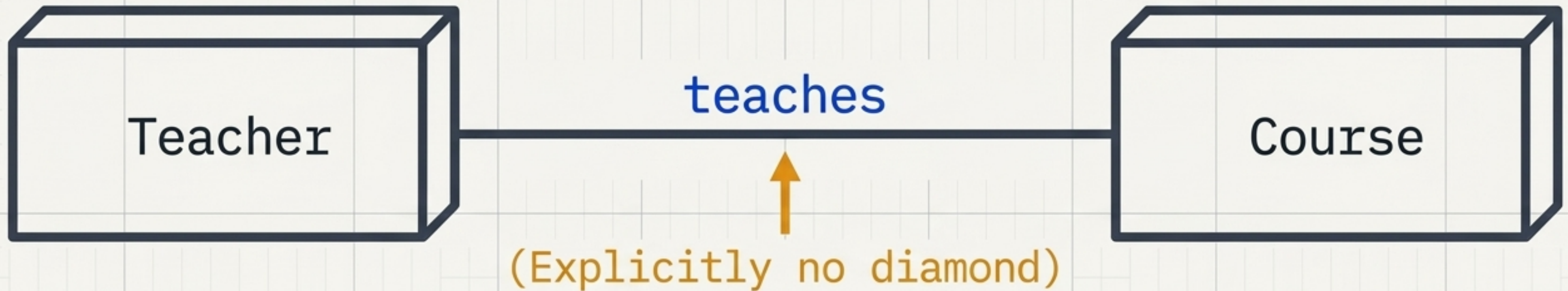


Object interactions are mapped using specific UML relationship notations



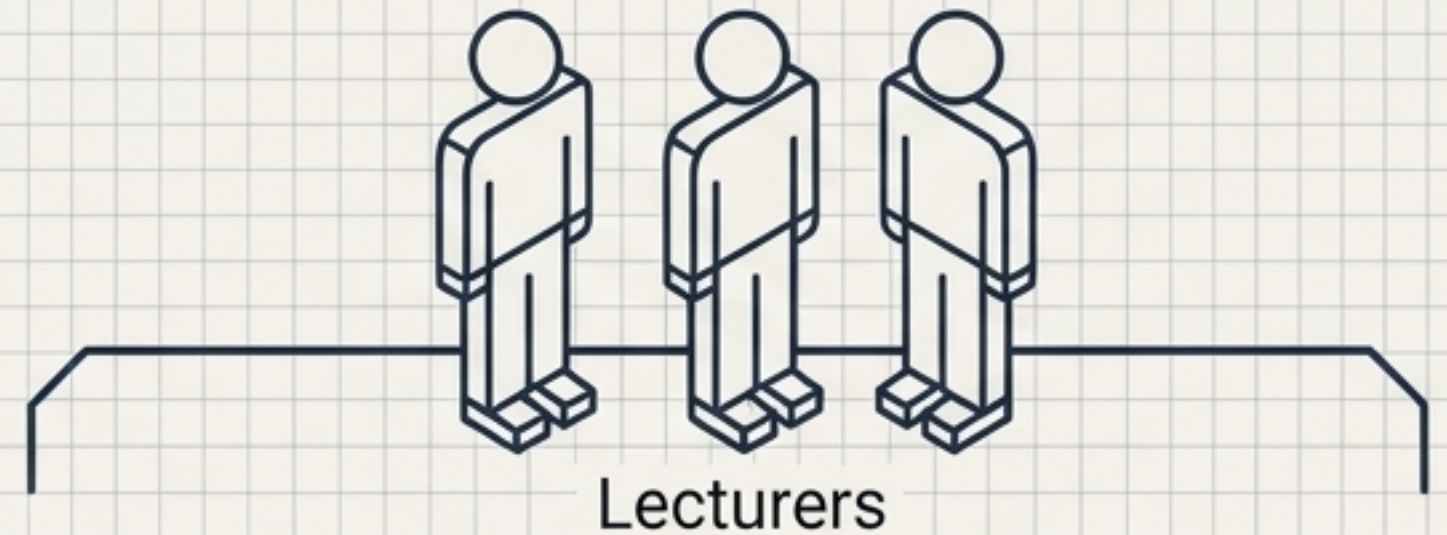
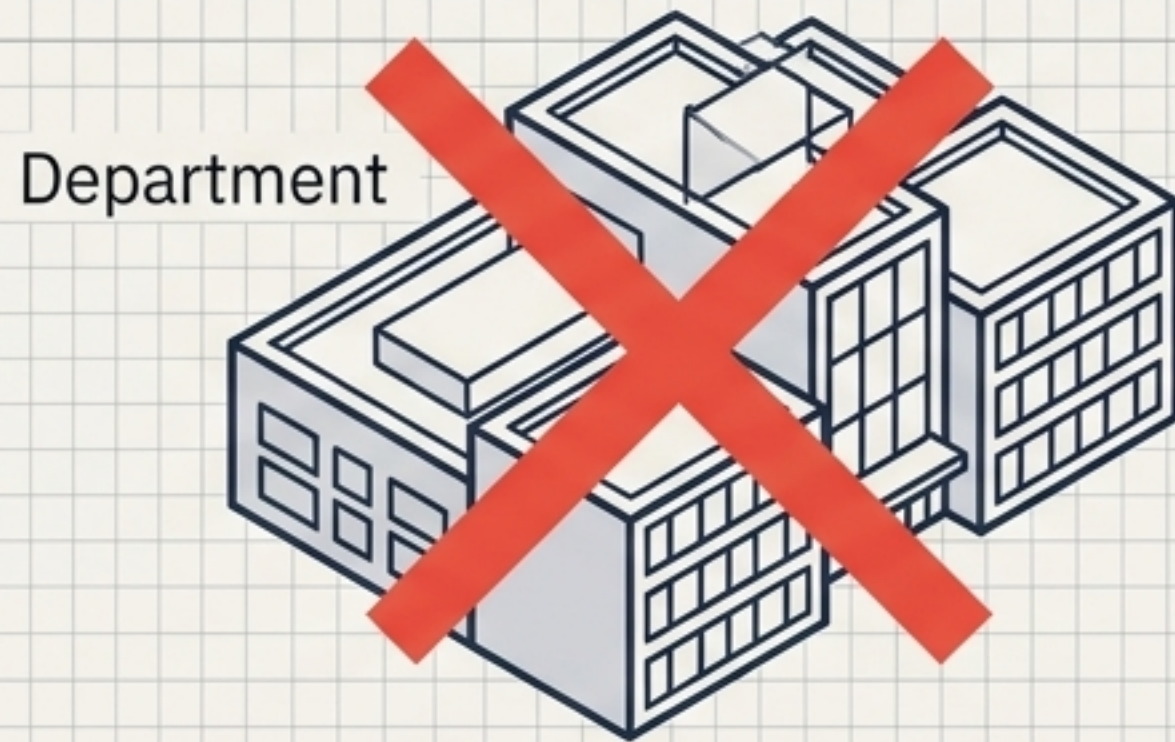
In design, we map interactions as relationships. Mastering these specific diamond symbols is critical for the **Continuous Assessment Test (CAT)**.

Association represents a general “knows about” relationship without ownership



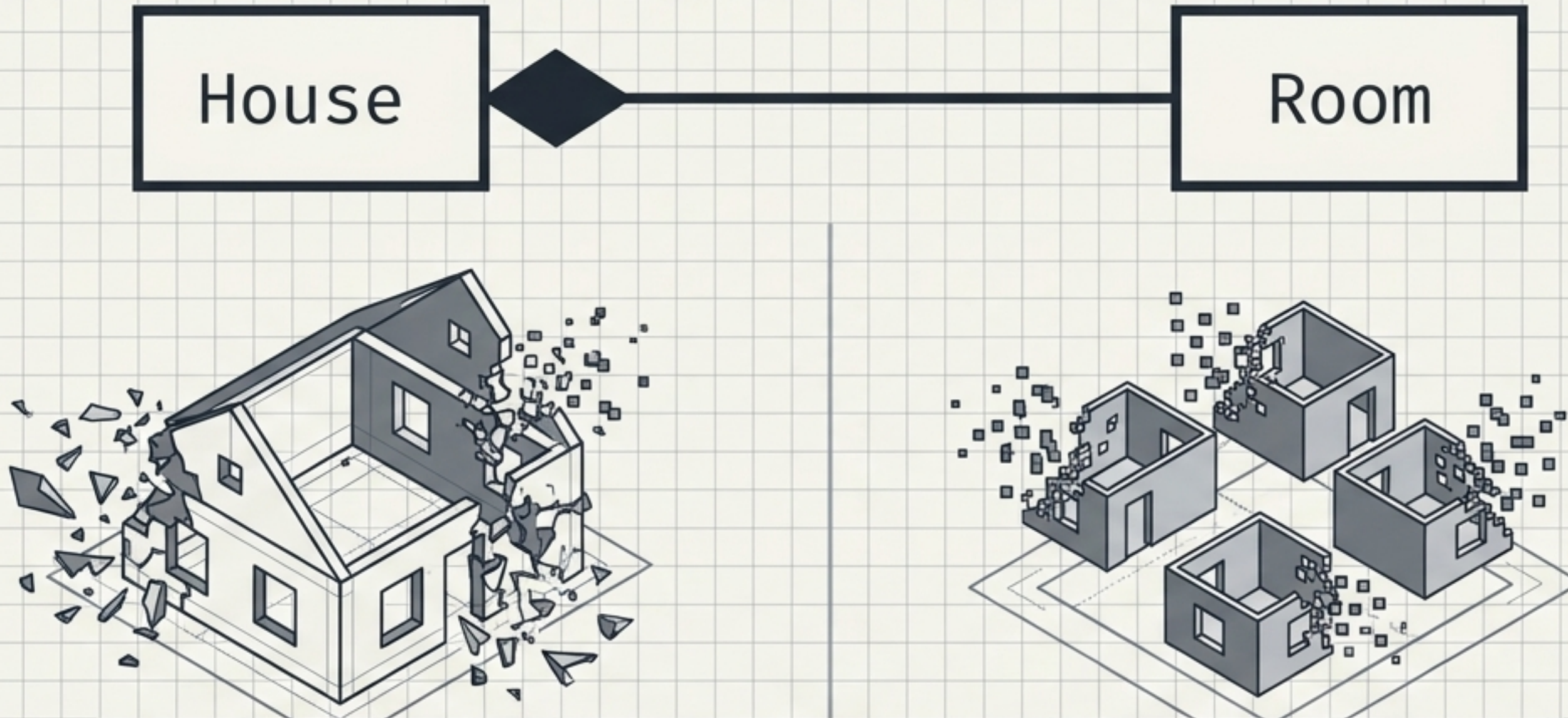
- Classes are at the same hierarchical level.
- One object uses or simply knows about another.
- No ownership or dependency is implied.

Aggregation defines a 'has-a' relationship where parts can exist independently



The part can exist independently of the whole. If the Department closes, the Lecturers survive and move elsewhere.

Composition forms a strict “has-a” relationship where parts die with the whole



The part cannot exist without the whole.
If the House is demolished, the Rooms no longer exist.

SOEN 201 Quick Revision Matrix: Core Concepts and UML Notations

Core Concepts Checklist

Class = Blueprint / **Object** = Concrete instance

Attributes = Data / **Methods** = Behaviour

Abstraction = Hides complexity /
Encapsulation = Protects data

Polymorphism = Same message, different
behaviour

Inheritance = "Is-a" hierarchy

The Diamond Diagnostic

	Visual	Name	Definition
1		Association	Uses / knows about
2		Aggregation	Has-a (part can live alone)
3		Composition	Has-a (part dies with the whole)